

OPTAM-MPC

APC Engineer User Manual

Consolidated Foundation v0.6.0

September 2026 Freeze

DRAFT 1 • FOR APC-ENGINEER FIELD TESTING

Practical configuration, validation, simulation and OPC reference

How to use this manual

Start with **Read this first**, then follow sections 2 to 14 in order. Use section 15 only when something fails. Section 17 defines the clean-start field test for this Draft 1 manual.

Contents

1. The frozen foundation
2. Decide which starter to use
3. Responsibilities
4. Install a starter package
5. Create a clean application workspace
6. Define the control problem
7. Understand the application files
8. Engineer a linear application
9. Engineer a nonlinear application
10. Configure the independent digital twin
11. Validate before running
12. Run offline
13. Tune in a controlled sequence
14. Run through OPC
15. Troubleshooting
16. Application acceptance record
17. Test this manual
18. Reference applications
19. Deliberate exclusions
20. Path forward

Purpose. This manual tells an APC engineer how to use the frozen OPTAM-MPC packages to create, validate and test an application. It is an operating and engineering reference. It is not a course in MPC theory, process modelling or Python programming.

Read this first

There are two user packages:

- Use **Linear Starter v0.6.0** when your process model can be expressed with gains, first-order lags, dead times and integrators. The APC engineer does not write Python application code.
- Use **Nonlinear Starter v0.2.0** when the prediction model needs nonlinear equations. The engineer supplies one small Python model file, but does not write the optimiser, OPC interface, simulator, plotting or reporting software.

The third archive, **Canonical Source v0.6.0**, is for software maintainers. An APC engineer creating an application normally does not use or edit it.

The quickest safe route is:

1. Choose linear or nonlinear.
2. Extract that starter into a new writable folder.
3. Run `setup_windows.bat` once.
4. Copy only the generic templates into a new application folder.
5. Engineer the model and configuration.
6. Validate before every run.
7. Run offline and inspect numerical acceptance results.
8. Test through the local OPC UA digital twin.
9. Preserve the application files and reports used for acceptance.

Important. A reported PASS means that the configured scenario passed the checks you defined. It does not prove the process model is correct, the plant is controllable under every condition, or the controller is safe for a live plant.

1. The frozen foundation

The collective freeze name is:

OPTAM-MPC Consolidated Foundation v0.6.0 — September 2026 Freeze

It contains these immutable archives:

Archive	Intended user	SHA-256
OPTAM_MPC_Linear_Starter_v0.6.0.zip	APC engineer using a linear dynamic model	0e5411cc443f08b91885c07a368783f142459376e67f40c3b2c1083e82d697cb
OPTAM_MPC_Nonlinear_Starter_v0.2.0.zip	APC engineer supplying nonlinear equations	9e72487df34aa8ed54cb27d9d2685ece5ac50d674f9a41f10a9ff96fb9172863
OPTAM_MPC_Canonical_Source_v0.6.0.zip	Software maintainer	f03b2b183d84e516915351cc736570f6c5ba8d8f28635c2f2714255c878e5a36

Do not edit, rebuild or replace these ZIP files. Documentation improvements are separate releases. A software change requires a new software version.

1.1 Verify the files on Windows

From Command Prompt, run:

```
certutil -hashfile OPTAM_MPC_Linear_Starter_v0.6.0.zip SHA256
certutil -hashfile OPTAM_MPC_Nonlinear_Starter_v0.2.0.zip SHA256
certutil -hashfile OPTAM_MPC_Canonical_Source_v0.6.0.zip SHA256
```

The displayed values must match the table above. A different hash means the file is not the frozen release.

1.2 What was verified

- 74 linear product tests, 31 nonlinear product tests and 4 canonical-core tests passed.
- Both 660-cycle heat-exchanger offline scenarios passed with zero solver failures.
- The 1051 linear surge-vessel OPC records exactly matched its offline records.
- The 661 nonlinear exchanger OPC records exactly matched its offline records.
- Release manifests and deterministic rebuilding passed.

These results establish a sound software baseline. They do not establish suitability for a particular plant.

2. Decide which starter to use

Use the simplest model form that adequately represents the control problem over the intended operating region.

Use the Linear Starter when

- Each MV-to-CV and measured-DV-to-CV relationship can be represented by a gain, lag, dead time or integrator.
- Linear superposition is acceptable over the operating range.
- A local linearisation of a nonlinear plant is adequate.
- You want a configuration-only application with minimal Python knowledge.

A single-input/single-output FOPDT model is simply a one-CV, one-MV case of this same linear engine. It is not a different controller.

Use the Nonlinear Starter when

- Gains change materially with operating point.
- Physical mixing, valve, heat-transfer, reaction or constraint relationships need explicit nonlinear equations.
- A linear model would be misleading across the required operating range.
- The engineer can review and test a small Python model function.

Stop before choosing either starter when

- Important dynamic states are unmeasured and require a state estimator.
- The model requires functionality excluded by the package contract.
- The plant interface or fallback requirements are not defined.
- The process model has not been independently reviewed.

Do not disguise an unmeasured state as a measured CV merely to fit the current interface.

3. Responsibilities

APC engineer

- Define CVs, MVs, measured DVs, units and operating direction.
- Supply a working dynamic model from first principles or another trusted source.
- Define model validity ranges, initial conditions and equilibrium.
- Select horizons, scaling and objective weights.
- Define independent model checks, scenarios and acceptance criteria.
- Review every configuration and result before escalation toward a plant trial.

OPTAM-MPC package

- Validate the supported configuration contract.
- Construct and solve the velocity-form MPC problem.
- Enforce configured MV position and move limits in the optimiser.
- Apply soft predicted CV-limit penalties.
- Run offline simulations and the local OPC UA digital twin.
- Produce trends, CSV data and machine-readable reports.

Site engineering and operations

- Validate instruments, actuators, units, scaling and tag mappings.
- Define safe manual, fallback and shutdown behaviour.
- Maintain alarms, trips, interlocks and independent protection.
- Approve commissioning procedures and live-plant authority.

Safety boundary. OPTAM-MPC protected CV limits are optimisation objectives, not safety trips. The frozen packages are engineering-development software and are not certified plant-control or safety systems.

4. Install a starter package

Requirements: Windows, Python 3.11 or later, internet access during initial dependency installation, and a writable working folder.

1. Extract the selected starter ZIP into a new folder, for example C:\optam_mpc_linear_v060.
2. Open Command Prompt in that folder.
3. Check Python:

```
python --version
```

1. Install the package environment once:

```
setup_windows.bat
```

The setup creates `.venv` and installs pinned dependencies. Use the supplied batch files; manual virtual-environment activation is not normally required.

Do not double-click the batch files. Run them from an open Command Prompt so errors remain visible. Quote any path containing spaces.

5. Create a clean application workspace

Do not edit the generic templates in place. Copy them into a new application folder.

Linear application

```
mkdir applications\my_application  
copy application_template.toml applications\my_application\application.toml  
copy simulation_template.toml applications\my_application\simulation.toml  
copy opc_template.toml applications\my_application\opc.toml
```

Nonlinear application

```
mkdir applications\my_application  
copy application_template.toml applications\my_application\application.toml  
copy model_template.py applications\my_application\model.py  
copy simulation_template.toml applications\my_application\simulation.toml  
copy opc_template.toml applications\my_application\opc.toml
```

The templates deliberately contain invalid placeholders. Replace every **REPLACE** value. Delete complete repeated blocks that are unnecessary and copy complete blocks when more variables or paths are required.

The completed applications under **examples** are reference solutions. Your application must not depend on them. During a clean-start test, use them only after making your own attempt.

6. Define the control problem before editing files

Write a short design basis containing:

1. **Objective:** what operational outcome should improve?
2. **CVs:** measured or calculated variables the MPC should control.
3. **MVs:** variables the MPC may manipulate.
4. **Measured DVs:** measured variables that affect prediction but cannot be manipulated.
5. **Hidden disturbances:** disturbances used in the test plant but not given to the controller.
6. **Limits:** operating limits, actuator limits and move limits.
7. **Model source:** first-principles calculation, trusted simulator or externally supplied model.
8. **Sample time and horizons:** with their durations in seconds or minutes.
9. **Scenarios:** disturbances, target changes and model mismatch cases.
10. **Acceptance:** numerical pass/fail criteria agreed before tuning.

Keep all units explicit. OPTAM-MPC does not perform automatic unit conversion.

7. Understand the four application files

File	Purpose	Engineer-owned content
application.toml	Controller structure, model and tuning	CVs, MVs, DVs, limits, model parameters and checks
model.py	Nonlinear prediction equations	Nonlinear applications only
simulation.toml	Independent test plant and acceptance scenario	Plant assumptions, disturbances, target changes, plots and checks
opc.toml	Local OPC UA test arrangement	Endpoint, mappings and timing

Do not put a disturbance into the measured-DV list unless it will genuinely be measured and supplied to the controller.

8. Engineer a linear application

8.1 Application and controller settings

Set:

```
[application]
name = "Your application name"
sample_time_seconds = 5.0
model_type = "linear_paths"

[controller]
prediction_form = "velocity"
prediction_horizon_samples = 30
control_horizon_samples = 10
maximum_solver_cpu_seconds = 4.0
```

Horizon values are samples. Multiply by sample time to understand their real duration. The control horizon may not exceed the prediction horizon.

8.2 Controlled variables

Each `[[cvs]]` block defines one instrumented process CV. Use either a target or a zone, never both.

```
[[cvs]]
id = "level"
name = "Vessel level"
unit = "%"
source_type = "instrumented_process"
initial = 50.0
target = 50.0
weight = 1.0
normalization = 10.0
protected_low = 20.0
protected_high = 80.0
protected_limit_weight = 1000.0
protected_limit_normalization = 1.0
display = "operator_and_engineering"
```

`normalization` is the error magnitude used for scaling. Weights express relative priority after scaling. Protected limits remain soft constraints.

8.3 Internal rate-of-change CV

For an integrating surge vessel, a filtered level-rate CV can supply useful proportional-like action while the primary level returns slowly toward its midpoint.

```
[[derived_cvs]]
id = "level_rate"
name = "Filtered level rate"
unit = "%/min"
source_type = "derived_auxiliary"
calculation = "rate_of_change"
source_cv = "level"
```

```

filter_alpha = 0.30
initial = 0.0
target = 0.0
weight = 0.01
normalization = 1.0
display = "engineering"

```

This is a controller CV even though it is internal. It belongs in the engineering contract and tuning record, but need not be displayed to the operator.

8.4 Manipulated variables

```

[[mvs]]
id = "outlet"
name = "Outlet flow"
unit = "m3/h"
initial = 60.0
lower = 20.0
upper = 100.0
maximum_increase_per_sample = 0.25
maximum_decrease_per_sample = 0.25
move_weight = 1.0
move_normalization = 1.0

```

The initial value must lie inside the limits. Position and move limits are hard optimiser constraints. Site actuator protection remains independent.

8.5 Measured disturbances

```

[[measured_disturbances]]
id = "inlet_temperature"
name = "Inlet temperature"
unit = "degC"
initial = 40.0

```

A measured DV affects a CV only through a model path.

8.6 Linear model paths

Add one `[[model_paths]]` block for each relationship included in the model.

Integrator:

```

[[model_paths]]
source = "outlet"
destination = "level"
type = "integrator"
integrating_gain = -0.0041666667
dead_time_seconds = 0.0
engineering_basis = "Vessel cross-sectional area and unit conversion"

```

`integrating_gain` uses destination units per minute per source unit.

First-order plus dead time:

```
[[model_paths]]
source = "hx_valve"
destination = "temperature"
type = "first_order"
gain = 0.04686745
time_constant_seconds = 45.0
dead_time_seconds = 5.0
engineering_basis = "Linearisation at the design operating point"
```

Static gain:

```
[[model_paths]]
source = "pressure"
destination = "flow"
type = "gain"
gain = 15.0
dead_time_seconds = 0.0
engineering_basis = "Local steady-state calculation"
```

Dead time must be a whole multiple of the sample interval. Missing paths mean zero modelled effect.

8.7 Optional combined-MV move penalties

Use named `[[move_combinations]]` blocks when simultaneous MV movement has a meaningful common or differential direction. Supply one coefficient per MV, in declared MV order.

```
[[move_combinations]]
name = "common valve movement"
coefficients = [0.5, 0.5]
weight = 1.0
normalization = 1.0

[[move_combinations]]
name = "differential valve movement"
coefficients = [0.5, -0.5]
weight = 1.0
normalization = 1.0
```

This facility is dimension-independent. For three MVs, provide three coefficients. If no combined objective is needed, omit the blocks.

8.8 Independent model implementation checks

Add checks derived independently from the configured model implementation.

Integrator check:

```
[[model_checks]]
source = "outlet"
destination = "level"
step = 1.0
expected_rate_per_minute = -0.0041666667
tolerance = 1.0e-9
```

For a non-integrating path use `metric = "steady_change"` and `expected_change`. To check dynamics, use `metric = "step_response"`, `time_seconds` and `expected_change`.

These checks confirm that the supplied model was implemented as intended. They are not model identification and do not prove the model represents the plant.

9. Engineer a nonlinear application

The TOML CV, MV, DV, limit, horizon, scaling and move-combination concepts remain the same. The important difference is that the engineer supplies the discrete nonlinear transition in `model.py`.

9.1 Supported model contract

The model must provide:

`x_next = F(x, u, d, parameters, dt_seconds)`

In this starter, every state is an instrumented primary CV. Hidden dynamic states and state estimation are not supported by this small interface.

9.2 Engineer-owned parts of `model.py`

Edit only:

1. `STATE_IDS`, `MV_IDS` and `DV_IDS`.
2. `validate_parameters(p)`.
3. `transition(x, u, d, p, dt)`.

The ID order must agree with `application.toml`.

```
import casadi as ca

STATE_IDS = ("flow", "temperature")
MV_IDS = ("hx_valve", "bypass_valve")
DV_IDS = ("inlet_temperature", "upstream_pressure")

def transition(x, u, d, p, dt):
    equilibrium = p["gain"] * u["hx_valve"] + d["inlet_temperature"]
    a = ca.exp(-dt / p["tau_seconds"])
    return {
        "flow": x["flow"],
        "temperature": a*x["temperature"] + (1-a)*equilibrium,
    }
```

Return absolute next-sample state values, not derivatives, increments or corrections. The core supplies velocity-form feedback.

9.3 Python rules for symbolic compatibility

- Use ordinary arithmetic and CasADi functions such as `ca.exp`, `ca.sqrt`, `ca.log` and `ca.sin`.
- Do not use `float()`, NumPy conversion or `math.exp` on `x`, `u` or `d`.
- Do not use Python `if` comparisons on symbolic process values.
- Do not use random numbers, file access, networking or mutable history.
- Do not call the optimiser from the model.
- Keep the model deterministic and reviewable.

Python `if` statements are acceptable inside `validate_parameters`, where parameter values are ordinary numbers.

9.4 Parameters and domain

Declare numerical parameters and a finite validity range for every CV, MV and DV:

```
[model]
file = "model.py"

[model.parameters]
tau_seconds = 45.0
gain = 0.5

[model.domain]
flow = [40.0, 80.0]
temperature = [50.0, 80.0]
hx_valve = [35.0, 65.0]
bypass_valve = [35.0, 65.0]
inlet_temperature = [35.0, 45.0]
upstream_pressure = [4.0, 6.0]
```

Use the region in which the equations are physically justified. Do not widen a domain merely to hide a validation failure. Denominators must remain away from zero and square-root arguments must remain valid throughout prediction.

The supplied nonlinear exchanger uses a bounded heat-transfer relationship based on $\exp(-A/q)$ within a validated positive-flow domain. It does not inherit the unbounded effectiveness expression found in the historical frozen demonstration.

9.5 Initial equilibrium

Initial CVs, MVs and DVs must form a stationary point of the controller model and configured test plant. An unexplained startup transient usually indicates inconsistent initial conditions, parameters or units.

9.6 Nonlinear checks

```
[[nonlinear_checks]]
name = "Cold inlet - independent next sample"
state = [60.0, 64.0]
mvs = [50.0, 50.0]
dvs = [37.0, 5.0]
expected_next = [60.0, 63.78849442465227]
tolerance = 1.0e-8
```

Add checks for equilibrium, each MV, each DV and important combinations or operating points. Calculate expected values independently; do not simply copy output from the function being tested.

10. Configure the independent digital twin

`simulation.toml` is deliberately separate from the controller model. It defines the test plant, scenario, plots and acceptance checks.

For linear applications, configure `[[plant_outputs]]`, `[[plant_inputs]]` and `[[plant_paths]]`. The plant may have different gains, lags or disturbances from the controller model.

For nonlinear applications, configure `[plant_model]` and its parameters/domain. Use an independently developed plant model when available. Reusing the same equations with different parameters tests implementation and mismatch handling, not independent physical accuracy.

10.1 Input roles

- mv**: driven by the controller.
- measured_dv**: changed externally and supplied to the controller.
- hidden**: changed externally but not supplied to the controller; available in the linear simulator.

Every controller CV, MV and measured DV must have exactly one plant mapping. Units and nominal MV/DV values must agree.

10.2 Events

An event changes a plant input at the beginning of a zero-based sample:

```
[[events]]
sample = 30
input = "wild_inlet"
value = 70.0
```

At a 5-second sample time, sample 30 starts at 150 seconds. The controller does not receive advance knowledge of future events.

Target events apply only to target-based primary CVs:

```
[[target_events]]
sample = 240
cv = "temperature"
value = 66.0
```

10.3 Acceptance checks

Define checks before tuning. Available metrics include:

Metric	Pass condition
minimum	Every selected record is at or above the limit
maximum	Every selected record is at or below the limit
max_abs_move	Largest consecutive movement is at or below the limit
final_error	Error at the final selected record is at or below the limit
max_abs_error	Largest error in the selected window is at or below the limit

Use `start_sample` and `end_sample` for a settling window. Use `max_abs_error`, not only a final point, when oscillation is a concern.

Check all important CV limits, MV positions, MV moves and required tracking performance. Do not weaken acceptance limits merely to obtain PASS.

11. Validate before running

Linear

```
validate_application.bat applications\my_application\application.toml  
applications\my_application\simulation.toml applications\my_application\opc.toml
```

Nonlinear

Use the same command and three arguments.

Validation checks syntax, IDs, dimensions, mappings, model checks, domains, initial equilibrium and a nominal solver move where applicable. It does not connect to a live plant.

Expected final result:

```
Validation and model checks: PASS
```

Treat every validation failure as useful information. Correct the engineering input; do not bypass the validator.

12. Run offline

```
run_simulation.bat applications\my_application\application.toml applications\my_application\simulation.toml
```

Results are written next to simulation.toml:

```
applications\my_application\results\results.png  
applications\my_application\results\results.csv  
applications\my_application\results\report.json
```

Expected final result:

```
Closed-loop acceptance: PASS
```

Exit code 0 means pass, 1 means a configuration or file error, and 2 means the simulation completed but acceptance failed.

Review `report.json`, not only the graph. It contains unrounded numerical checks, solver failures and provenance hashes. Generate a readable record with:

```
.venv\Scripts\python write_engineering_results.py applications\my_application\results\report.json  
applications\my_application\ENGINEERING_RESULTS.md
```

Preserve the application, model, simulation, OPC configuration and report together. Repeating a run replaces the normal `results` files, so archive an accepted run before further changes.

13. Tune in a controlled sequence

Use engineering judgement rather than blind numerical searching.

1. Check signs, units, equilibrium and model response first.
2. Confirm feasibility and MV authority.
3. Choose meaningful CV and MV normalizations.
4. Start with moderate target weights and sufficient move suppression.
5. Prioritise protected limits strongly, remembering they are soft.
6. Tune one objective group at a time.
7. Repeat all acceptance scenarios after every material change.
8. Test model mismatch and measurement disturbance sensitivity.

Increasing a CV weight relative to others generally increases its priority. Increasing an MV move weight discourages movement and normally trades response speed for smoothness and robustness.

The consolidated core can change CV weights, CV-rate weights, targets, individual MV move weights and declared move-combination weights as runtime parameters without rebuilding the NLP. Online adjustment does not remove the need for controlled authority, audit records and retesting.

14. Run through the local OPC UA digital twin

This stage verifies the two-process interface. It is not a live-plant connection.

Window 1 — digital twin

```
run_digital_twin.bat applications\my_application\application.toml  
applications\my_application\simulation.toml applications\my_application\opc.toml
```

Wait for:

```
Twin READY
```

Window 2 — controller

```
run_controller.bat applications\my_application\application.toml applications\my_application\opc.toml
```

The twin owns simulated time, publishes one coherent measurement snapshot, waits for a valid complete MV vector and then advances one interval. On a controller or communication fault, the test twin freezes rather than continuing. This behaviour is intentional for reproducible testing and must not be copied blindly as a live-plant fallback philosophy.

Both processes must use identical application contents. A fingerprint mismatch is rejected. Stop both processes before editing configuration files.

OPC results are written to `opc_results`. Compare offline and OPC results before acceptance. The verified reference cases achieved exact CSV equality.

15. Troubleshooting

Python is not found

Install Python 3.11 or later, make sure `python --version` works, then rerun `setup_windows.bat`.

A module is missing

Setup did not complete in this extracted package. Rerun `setup_windows.bat` and inspect the first error.

File not found

Check the working directory and all arguments. Put paths containing spaces in double quotes.

TOML parsing fails

Check `[table]` versus `[[repeated_table]]`, quotation marks, commas in arrays and duplicate keys.

Unknown ID or mapping

Use controller IDs in model paths and `controller_id` mappings. Use plant IDs in plant events and plant paths. IDs are case-sensitive.

Model validation fails

Check parameter names, ID order, units, equilibrium, domain endpoints, denominators and symbolic compatibility. Do not widen the domain or tolerance without a physical reason.

Solver fails

Check model finiteness, feasibility, scaling, MV authority, horizons and operating domain. A simulation holds the last MV after a solver failure and records it; that is a development fallback only.

Acceptance fails

Inspect the exact failed check and relevant CSV window. Review the model, objective conflict, scaling, constraints, mismatch and scenario. Do not simply relax the failed criterion.

OPC does not start

Start the twin first, confirm the endpoint/port is free, use the same `opc.toml` in both windows and check that both processes use identical application contents.

16. Application acceptance record

Before declaring an application ready for the next engineering stage, retain:

- Design basis and model source.
- Exact starter archive name and hash.
- `application.toml` and, for nonlinear applications, `model.py`.
- `simulation.toml` and `opc.toml`.
- Independent model-check calculations.
- Offline `report.json`, CSV, plot and generated engineering summary.
- OPC report and offline/OPC comparison.
- Tuning rationale and all changed values.
- Known limitations and untested operating regions.
- Reviewer names, review date and decision.

Plant deployment requires a separate site-specific lifecycle covering cyber security, live tag mapping, execution deadlines, fallback behaviour, permissions, change management, commissioning and operations approval. That lifecycle is outside this freeze.

17. How to test this manual

The purpose of Draft 1 is to discover where an APC engineer cannot proceed confidently.

Test A — linear surge vessel

1. Use the Linear Starter and generic templates only.
2. Create the single surge-vessel application.
3. Validate and run it offline.
4. Run it through OPC.
5. Only then compare with `examples/01_surge_vessel`.

Test B — linear 2x2 heat exchanger

Repeat the clean-start exercise for two CVs, two MVs and two measured DVs. Include gains, lags, dead times, interaction and feedforward. Compare afterward with `examples/02_heat_exchanger`.

Test C — nonlinear heat exchanger

Use the Nonlinear Starter and generic templates. Supply `model.py`, validate domains and independent next-step checks, then run offline and through OPC. Compare afterward with `examples/01_heat_exchanger`.

Record feedback under these headings

- Step where progress stopped.
- Exact instruction that was unclear.
- Error message or unexpected result.
- Assumption the tester had to make.
- Whether the completed example resolved the problem.
- Python help required, if any.
- Suggested wording or software improvement.
- Severity: blocking, serious, inconvenient or cosmetic.

Do not change the frozen software during this trial. Classify each finding as documentation, example, usability, defect or requested enhancement. Improve the manual separately; create a new software version only for confirmed software changes.

18. Reference applications

Linear surge vessel

One integrating vessel, wild inlet disturbance and manipulated outlet flow. Objectives: respect level limits, return level slowly toward 50% to restore surge capacity, and minimise outlet-flow movement. The internal filtered level-rate CV targets zero.

Linear heat exchanger with bypass

Two CVs: combined outlet flow and temperature. Two MVs: exchanger-line and bypass-line valves. Two measured DVs: inlet temperature and upstream pressure. The model uses linear gains, first-order lags and whole-sample dead times.

Nonlinear heat exchanger with bypass

The same broad control problem with modest valve curvature, pressure-dependent flow, nonlinear heat transfer and mixing. Branch flows are calculated internally and are not assumed measured.

19. What this manual deliberately does not do

This manual does not:

- Teach MPC mathematics from first principles.
- Teach process modelling or model identification.
- Make an APC engineer a Python programmer.
- Define a universal live-plant OPC driver.
- Provide a state estimator for hidden dynamic states.
- Replace hazard analysis, regulatory controls or safety systems.
- Prove that a model or tuning is suitable for a real plant.

Those topics may later become tutorials, engineering tools or new software phases. They should be chosen from evidence gathered during the manual trial.

20. Path forward after Draft 1

1. Freeze and preserve the exact software foundation.
2. Have Pieter and external APC resources perform Tests A, B and C.
3. Consolidate feedback without changing the frozen ZIPs.
4. Revise the user manual where findings are documentation-related.
5. Log genuine software defects and enhancements for a new version.
6. Decide from evidence whether the next priority is a tutorial, improved templates, an application wizard, additional model interfaces or an offline AI/RAG assistant.

The immediate success criterion is simple: **Can a competent APC engineer use the generic materials to build and test an application without hidden help from application-specific software?**